

Application of TCN and Transformer Networks for Large-Scale System Log Anomaly Detection

Hoang-Tu Trinh

*Department of Information Technology
Ho Chi Minh City University of
Foreign Languages - Information Technology
(HUFLIT)*

Ho Chi Minh City, Vietnam
Email: tht.csec2005@gmail.com

Tien-Thanh Cao

*Department of Information Technology
Ho Chi Minh City University of
Foreign Languages - Information Technology
(HUFLIT)*

Ho Chi Minh City, Vietnam
Email: thanhct@hufit.edu.vn

Abstract—This paper presents a large-scale system log anomaly detection pipeline based on Drain3 and event sequence analysis, comparing Temporal Convolutional Networks (TCN) and Transformers against traditional baselines on two large-scale system log datasets. Raw logs are parsed into event templates, grouped into sequences, and scored for anomalies using five branches: PCA/SVD, Isolation Forest, DeepLog (LSTM next-event prediction), TCN (Temporal Convolutional Network), and Transformer masked event modeling. On HDFS (500k lines, 137 templates), experiments across 5 random seeds show that PCA achieves an F1-score of 0.5711 ± 0.0944 , slightly outperforming DeepLog and TCN (0.4613 ± 0.0199). On BGL (500k lines, 198 templates, window size $W=100$), TCN leads with an F1-score of 0.9826 ± 0.0022 (top-k mode) and 0.9474 ± 0.0035 (NLL threshold mode), outperforming DeepLog LSTM (0.9820 ± 0.0042 and 0.9381 ± 0.0075) as well as the val-tuned Transformer (0.9388 ± 0.0080). Under the tested configuration, TCN demonstrates a notable classification advantage on BGL and competitive inference latency compared to LSTM and Transformer models, while PCA remains competitive on short-sequence datasets such as HDFS. The contributions include: (i) a reproducible evaluation pipeline that prevents data leakage; (ii) a cross-dataset evaluation between HDFS and BGL; (iii) a grouping strategy study on fixed and sliding windows; (iv) ablation studies on Drain3 threshold, top-k, and mask ratio; and (v) inference-latency and qualitative explainability analyses.

Index Terms—Log anomaly detection, masked event modeling, Transformer, Drain3, AIOps, distributed systems.

I. INTRODUCTION

Modern large-scale computing infrastructures, including distributed storage systems, supercomputers, web services, and cloud platforms, generate logs with diverse formats and frequencies. These system logs capture both the operational state and the behavioral patterns of running services, while manually crafted detection rules quickly become obsolete as the infrastructure evolves. Effective log anomaly detection therefore requires analysis at the event sequence level rather than individual log lines.

The complete source code and raw training logs to ensure experiment reproducibility are publicly available at: <https://github.com/thtsec/vnict2026-log-anomaly>.

This paper is positioned as an applied research and system evaluation study, rather than proposing a new anomaly detection algorithm. The central idea is to treat the event template sequence after Drain3 parsing as an intermediate representation layer: compact enough to run lightweight baselines, structured enough to extend to sequence models, and clear enough to map to operational units such as sessions, anonymized users, or time windows. Three research questions are addressed:

- **RQ1:** How does the method of constructing event sequences after Drain3 affect the anomaly detection signal?
- **RQ2:** How do the five scoring branches (reconstruction, isolation, next-event prediction, causal convolution, and masked event modeling) compare on the same event sequence?
- **RQ3:** Does the performance ranking of these branches remain consistent when moving from HDFS to BGL (datasets with very different characteristics)?

The main contributions include: (1) an end-to-end pipeline of Drain3 parsing, event sequencing, and anomaly scoring with five alternative branches (PCA/SVD, Isolation Forest, DeepLog, TCN, and Transformer masked event modeling); (2) a multi-seed evaluation (5 random seeds) on HDFS with a protocol that prevents test set leakage, along with a grouping strategy study for Block ID, fixed window, and sliding window; (3) a cross-dataset HDFS-BGL evaluation on the same pipeline; (4) an ablation study for Drain3 threshold, DeepLog top-k, and Transformer mask ratio; (5) an inference latency benchmark, expected error analysis, and qualitative explainability case study for deployment-oriented interpretation.

II. RELATED WORK

The earliest log anomaly detection methods typically represented log data as event count vectors or feature matrices, and then applied traditional machine learning models such as PCA, clustering, or Isolation Forest [10], [11]. PCA is used to separate normal behavior space and detect points deviating from the principal subspace. This group of methods is lightweight

and easy to deploy, but usually ignores the temporal order and contextual relationships between log events.

To address this limitation, sequential models like DeepLog [1] use LSTM networks to predict the next event based on previous events. LogAnomaly [13] and LogRobust [14] extend this direction by combining sequential and semantic log information to deal with unstable log data. Additionally, semi-supervised approaches such as PLELog [15] use probabilistic label estimation to reduce dependence on manual labels. Overall, RNN/LSTM architectures capture sequence relationships better than traditional statistical methods, but still suffer from sequential processing limitations and are difficult to parallelize.

To represent unstructured logs for machine learning models, log parsing techniques are used to map each log line to a template. Drain [3] and Drain3 are widely used due to their fixed-depth parse tree structure, which allows high-speed online parsing. Recently, LogPPT [7] used prompt-based few-shot learning to improve parsing accuracy when labeled data is scarce.

The Transformer architecture with its self-attention mechanism [4] has been widely applied in AIOps. LogBERT [2] employs a Transformer encoder combined with masked event modeling to learn bidirectional representations for normal log sequences, improving contextual anomaly detection. Survey papers also note a strong shift from traditional sequential models to self-supervised deep learning based on Transformers [16]. In parallel, Large Language Model (LLM) and prompt learning approaches such as LogPrompt [8] and LogGPT [9] enhance semantic understanding, but face major barriers in terms of inference cost and real-time latency.

TABLE I
COMPARISON OF METHODOLOGY GROUPS AND RESEARCH GAPS

Group	Representatives	Key Limitation	Improvement in This Work
Statistical	PCA, IF [10], [11]	No temporal ordering	Lightweight baseline quantification
Sequential	DeepLog [1], [13]	Limited sequence length, bottleneck	Parallel TCN integration
Transformer	LogBERT [2]	Dependent on parsing quality	Decoupled parsing, grouping, and scoring

From existing works, four main research gaps can be identified: (i) traditional statistical methods do not learn the order of log events; (ii) sequential LSTM models are limited with long log sequences; (iii) Transformer models depend heavily on the parsing step; (iv) mapping anomaly detection pipelines to large networks requires clarifying the grouping strategy and business scenarios.

To address these gaps, this paper builds a standardized pipeline that separates three phases (parsing, grouping, and scoring) using Drain3 as a preprocessing step to clean the token space. We evaluate in parallel the scoring branches including lightweight baselines (PCA, TruncatedSVD, Isolation Forest), sequential LSTM (DeepLog), the integrated Temporal Convolutional Network (TCN), and Transformer

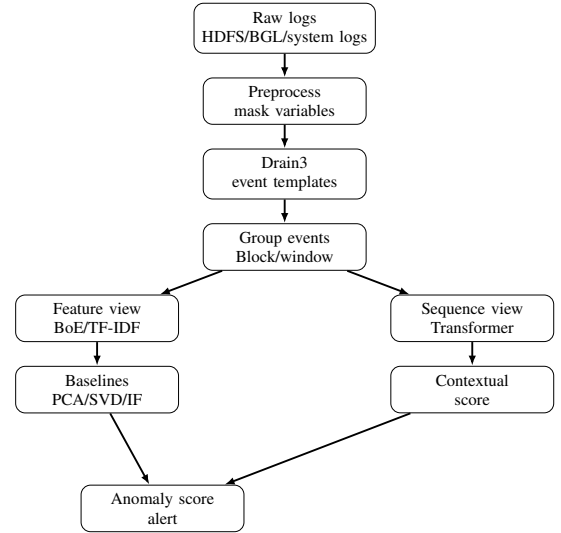


Fig. 1. Proposed log anomaly detection pipeline block diagram

masked event modeling. Performance is carefully measured using Precision, Recall, and F1-score averaged over 5 different random seeds to ensure scientific reliability. To ensure experiment reproducibility and verification, the complete source code and raw training logs are publicly available at <https://github.com/thtsec/vnict2026-log-anomaly>.

III. PROPOSED METHODOLOGY

A. Pipeline Overview

The proposed pipeline is designed to clearly separate three processing layers: log standardization, event sequence construction, and anomaly detection. The input is a raw log dataset $L = \{l_1, l_2, \dots, l_N\}$ generated from the operating system. The output is an anomaly label or score $a(S)$ for each event sequence S . In HDFS experiments, a sequence corresponds to a Block ID; in BGL experiments, a sequence corresponds to a fixed-size time window. In broader deployment scenarios, a sequence can correspond to a login session, an anonymized user, a service, or a sliding time window.

The overall process consists of six steps:

- **Step 1 – Log Collection:** receive logs from source systems such as HDFS, BGL, or other structured system log sources.
- **Step 2 – Preprocessing:** normalize variable fields such as timestamps, IPs, user IDs, session IDs, and query parameters.
- **Step 3 – Log Parsing:** use Drain3 to map each log line to an event template and event ID.
- **Step 4 – Event Grouping:** group event IDs by Block ID or by session/time window to create event sequences.
- **Step 5 – Feature Representation:** generate bag-of-events/TF-IDF representation for baselines or keep the raw event sequence for Transformer masked event modeling.
- **Step 6 – Anomaly Detection:** compute anomaly scores using reconstruction error, Isolation Forest, or sequence learning models.

Beyond the HDFS and BGL benchmarks evaluated in this study, the pipeline architecture is designed to accommodate other structured log sources after appropriate preprocessing and anonymization. Instead of building specific rules for each log source, the system learns normal operational patterns from event sequences and flags sequences that deviate from these patterns. This generalizability requires further validation on production system logs in future work.

B. Preprocessing, Drain3 Parsing, and Event Grouping

The input is a raw log set $L = \{l_1, l_2, \dots, l_N\}$. The process consists of: (i) normalizing variable parameters (IP, timestamp, ID); (ii) parsing raw logs into event templates using Drain3 [3]; (iii) grouping event IDs into sequences by Block ID or time windows.

Drain3 uses a fixed-depth tree to find the corresponding event template for each log line. The similarity between two log lines T_1, T_2 is calculated as the ratio of matching tokens:

$$\text{Sim}(T_1, T_2) = \frac{1}{n} \sum_{i=1}^n \text{equ}(t_{1,i}, t_{2,i}) \quad (1)$$

If $\text{Sim}(T_1, T_2) > st$ (similarity threshold, default 0.5), the log line is assigned to an existing template; otherwise, Drain3 creates a new template. After this step, each template is assigned an event ID, and the log sequence is represented as:

$$S = [E_1, E_2, \dots, E_m] \quad (2)$$

For HDFS, the sequence S is grouped by Block ID. For BGL, it is grouped by non-overlapping windows of $W=100$ lines. We also support sliding window strategies to study the effect of the grouping strategy.

C. Lightweight Machine Learning Baselines

PCA and TruncatedSVD are applied to the bag-of-events vector representation $x = [\text{count}(E_j, S)]_{j=1}^d$ to learn the normal behavior subspace. The anomaly score is computed as the reconstruction error:

$$\text{score}(S) = \frac{1}{d} \sum_{j=1}^d (x_j - \hat{x}_j)^2 \quad (3)$$

where $\hat{x}$ is the reconstructed vector after projecting onto the low-dimensional space (the number of principal components is set to 20 by default). The anomaly threshold τ is chosen as the 95th percentile on the normal training set. A sequence S is labeled as anomalous if $\text{score}(S) > \tau$. Isolation Forest is applied to the TF-IDF feature representation of the event sequences. The model isolates sparse data points using random partitioning trees.

D. Deep Sequence Models and Transformer

DeepLog (LSTM): Replicated as next-event prediction using a 2-layer LSTM [1]. For each position i , the model predicts the probability of event E_i given the context of the previous g events:

$$\hat{P}(E_i | E_{i-g}, \dots, E_{i-1}) = \text{softmax}(W_o h_i + b_o) \quad (4)$$

In top-k mode, a sequence is flagged as anomalous if at least one actual event lies outside the top K predictions with the highest probability. In NLL threshold mode, the anomaly score

TABLE II
STATISTICS OF HDFS AND BGL DATASETS USED IN THE EXPERIMENTS

Metric	HDFS	BGL
Raw log lines	500,000	500,000
Grouping unit	Block ID	Window ($W=100$)
Number of sequences/windows	36,305	5,000
Number of normal sequences	34,558	2,657
Number of anomalous sequences	1,747	2,343 (47%)
Number of event templates (Drain3)	137	198
Drain3 similarity threshold θ	0.5	0.5
Drain3 tree depth	4	4

of the sequence is the average negative log-likelihood (NLL) of the events.

Temporal Convolutional Network (TCN): Integrated to improve parallelization and expand the receptive field. TCN utilizes dilated causal 1D convolutions [19]. The convolution operation on element s of the input sequence x with a filter f of size k and a dilation factor d is defined as:

$$F(s) = (x *_d f)(s) = \sum_{j=0}^{k-1} f(j) \cdot x_{s-d \cdot j} \quad (5)$$

A Chomp1d mechanism is used to crop the padding, ensuring causality. TCN uses the same dataset, scoring methods (top-k and NLL), and 95th percentile threshold selection as DeepLog to ensure a fair comparison.

Transformer (LogBERT-inspired): Employs a Transformer Encoder architecture combined with masked event modeling to learn bidirectional contexts [2]. A proportion of events (default 15%) are replaced by a [MASK] token, and the model learns to recover them using cross-entropy loss:

$$L_{MLM} = - \sum_{i \in M} \log P(k_i | S_{\text{masked}}) \quad (6)$$

where M is the set of masked positions. During inference, the anomaly score of sequence S is the average MLM loss over the masked positions. An anomaly decision is made by comparing this score with a threshold τ selected as the 95th percentile on the normal validation set.

IV. EXPERIMENTAL SETUP

A. Datasets

The study uses HDFS and BGL from the LogHub benchmark [6], [12]. HDFS contains logs from a Hadoop system and is labeled by Block ID (anomalous if the block contains errors). We use the first 500k lines for verification. BGL contains logs from a supercomputer and is grouped by non-overlapping windows of $W=100$ lines to assign anomaly labels. The statistics of the two datasets after Drain3 parsing are described in Table II. While HDFS and BGL serve as representative benchmarks for large-scale system logs, we acknowledge that they may not fully capture all the dynamic properties of live enterprise or industrial networks. Further validation on proprietary production logs remains an essential area for future exploration.

Execution Environment: The experiments were deployed on a computer configured with an Intel Core i7-13620H CPU, an RTX 4050 6GB GPU, 16GB RAM, running Ubuntu 22.04, Python 3.9, and PyTorch 2.0.

Experimental Procedure and Parameters: The data is split into train/test sets with a 70/30 ratio (baselines) or train/val/test with a 60/20/20 ratio (Transformer). PCA and TruncatedSVD are trained on normal sequences to select the 95th percentile threshold. The number of principal components is fixed at 20. Isolation Forest is configured with 200 decision trees on TF-IDF features. For the mini Transformer, the model consists of 2 encoder layers ($d_{model}=64$, 4 heads, feed-forward dimension 128) trained using the AdamW algorithm [17] for 5 epochs with a mask ratio $r=0.15$. The DeepLog and TCN baselines use an embedding size of 64, a hidden size of 64, optimized using Adam with a learning rate of 1×10^{-3} , and trained for 10 epochs. To evaluate reliability and prevent data leakage, all results are reported as the mean $\pm$ standard deviation over 5 random seeds (21, 42, 84, 123, 777).

Evaluation Metrics: We use Precision, Recall, and F1-score to evaluate performance:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2 \cdot P \cdot R}{P + R} \quad (7)$$

In actual network operations, a low Precision causes many false alarms, while a low Recall leads to missed incidents. Therefore, F1-score is the primary metric for comparing performance.

V. RESULTS AND DISCUSSION

A. Cross-Dataset Performance Comparison

The results in Table III reveal that no single model dominates across both datasets — a finding that underscores the importance of evaluating the entire pipeline rather than any individual algorithm. On HDFS, the lightweight baselines achieve competitive results: PCA obtains an F1-score of 0.5711 ± 0.0944 , outperforming deep learning models including DeepLog LSTM and TCN (both 0.4613 ± 0.0199). The mini Transformer obtains a low F1-score of 0.2950 ± 0.0693 . We note that the Transformer evaluated here is a lightweight variant optimized for low latency; a larger, more deeply tuned Transformer model might yield different classification results but at the cost of higher latency. This observation arises because HDFS is a highly structured log dataset with a small vocabulary (137 templates) and very short sequences (average length of 13.77 events). In such settings, anomalies are often manifested as distributional shifts in event frequencies, which are effectively captured by PCA reconstruction error on bag-of-events features. Conversely, deep sequence models do not benefit from their temporal modeling capacity due to sequence sparsity and the absence of complex contextual transitions.

Visual Analysis and Threshold Sensitivity: In addition to F1-score, the baselines are analyzed through ROC and Precision-Recall (PR) curves in Fig. 2. The results show that PCA and TruncatedSVD maintain better separability than Isolation Forest across most threshold ranges. For HDFS (which is highly imbalanced), the PR curves indicate that PCA/SVD maintain high Precision at high Recall levels, whereas Isolation Forest declines rapidly. The corresponding AUC-ROC scores for PCA, TruncatedSVD, and Isolation Forest on HDFS (with seed 42) are 0.7722, 0.7536, and 0.7136, respectively,

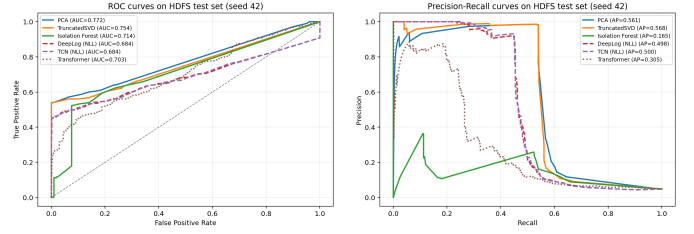


Fig. 2. ROC (left) and Precision-Recall (right) curves on HDFS.

and the Average Precision (AP) scores are 0.5613, 0.5676, and 0.1645, respectively.

In contrast, on BGL, the deep sequence models show strong results under the tested configuration: TCN (top-k) achieves the highest F1-score of 0.9826 ± 0.0022 , followed closely by DeepLog (0.9820 ± 0.0042). In NLL mode, TCN (0.9474 ± 0.0035) continues to outperform DeepLog (0.9381 ± 0.0075) and the Transformer (0.9388 ± 0.0080). Notably, the traditional baselines PCA and TruncatedSVD also perform well on BGL (0.9217 and 0.9356 F1-scores, respectively), indicating that BGL sequence structures are inherently separable. This cross-dataset divergence — where lightweight baselines prevail on HDFS while sequence models excel on BGL — highlights that the choice of scoring branch should be guided by dataset characteristics rather than model complexity alone.

Grouping Strategy Study (RQ1): To address RQ1, we evaluate the anomaly detection performance on BGL under four grouping configurations: non-overlapping fixed windows of size $W=50, 100, 200$, and a sliding window of size $W=100$ with stride 50. Table VI shows the F1-score results for PCA, DeepLog (top-k), and TCN (top-k). Because this study focuses on the relative effect of grouping configurations, all results in Table VI are reported under a single-seed controlled setting on a 200k BGL log subset (which has a different anomaly ratio) and should not be directly compared with the multi-seed aggregate results in Table III.

The results in Table VI indicate that the window size W directly impacts detection performance. For PCA, F1-score remains low at $W = 50$ and $W = 100$, but improves substantially at $W = 200$, suggesting that larger windows provide more stable frequency distributions for reconstruction-based scoring. Conversely, sequence models (DeepLog, TCN) show robust performance across all window configurations. Crucially, the sliding window configuration ($W=100$, stride = 50) yields the highest performance (DeepLog 0.9004, TCN 0.9032) by increasing training sample density and mitigating border-crossing boundary effects.

B. Parameter Sensitivity, Latency, and Error Analysis

To understand the role of the three key hyperparameters, we perform three ablation studies on HDFS 200k (with seed 42): (i) Drain3 similarity threshold θ , (ii) DeepLog top-k parameter K , and (iii) Transformer mask ratio r . Table IV summarizes the results.

The ablation results provide three concrete insights. (A) The Drain3 similarity threshold has a strong impact on the pipeline quality: $\theta=0.3$ leads to over-merging (only 25 templates,

TABLE III
CROSS-DATASET PERFORMANCE COMPARISON, REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER 5 RANDOM SEEDS.

Method	HDFS			BGL		
	Precision	Recall	F1	Precision	Recall	F1
PCA	0.6274 $\pm$ 0.1944	0.5450 $\pm$ 0.0055	0.5711 $\pm$ 0.0944	0.9473 $\pm$ 0.0113	0.8976 $\pm$ 0.0144	0.9217 $\pm$ 0.0098
TruncatedSVD	0.4298 $\pm$ 0.2555	0.5527 $\pm$ 0.0106	0.4540 $\pm$ 0.1366	0.9462 $\pm$ 0.0051	0.9255 $\pm$ 0.0228	0.9356 $\pm$ 0.0117
Isolation Forest	0.1726 $\pm$ 0.0146	0.1080 $\pm$ 0.0084	0.1328 $\pm$ 0.0103	0.1110 $\pm$ 0.0171	0.0248 $\pm$ 0.0045	0.0404 $\pm$ 0.0070
Transformer (unsup.)	0.2094 $\pm$ 0.0490	0.2982 $\pm$ 0.0892	0.2458 $\pm$ 0.0639	0.9422 $\pm$ 0.0110	0.9073 $\pm$ 0.0216	0.9242 $\pm$ 0.0085
Transformer (val-tuned)	0.5102 $\pm$ 0.1497	0.2365 $\pm$ 0.1156	0.2950 $\pm$ 0.0693	0.9841 $\pm$ 0.0017	0.8975 $\pm$ 0.0157	0.9388 $\pm$ 0.0080
DeepLog (top-k)	0.9862 $\pm$ 0.0201	0.3013 $\pm$ 0.0167	0.4613 $\pm$ 0.0199	0.9654 $\pm$ 0.0083	0.9991 $\pm$ 0.0012	0.9820 $\pm$ 0.0042
DeepLog (NLL)	0.2644 $\pm$ 0.0198	0.3601 $\pm$ 0.0256	0.3046 $\pm$ 0.0198	0.9408 $\pm$ 0.0066	0.9355 $\pm$ 0.0157	0.9381 $\pm$ 0.0075
TCN (top-k)	0.9862 $\pm$ 0.0201	0.3013 $\pm$ 0.0167	0.4613 $\pm$ 0.0199	0.9666 $\pm$ 0.0039	0.9991 $\pm$ 0.0012	0.9826 $\pm$ 0.0022
TCN (NLL)	0.2621 $\pm$ 0.0277	0.3644 $\pm$ 0.0291	0.3042 $\pm$ 0.0236	0.9412 $\pm$ 0.0085	0.9537 $\pm$ 0.0094	0.9474 $\pm$ 0.0035

TABLE IV
HYPERPARAMETER ABLATION STUDY ON HDFS 200K (SEED 42)

Configuration	Templates	Precision	Recall	F1
<i>(A) Drain3 θ, evaluated on PCA reconstruction</i>				
$\theta = 0.3$	25	0.3695	0.5215	0.4325
$\theta = 0.5$ (default)	68	0.4070	0.5550	0.4696
$\theta = 0.7$	2455	0.1000	0.0287	0.0446
<i>(B) DeepLog top-k</i>				
$K = 1$	68	0.0466	0.6403	0.0869
$K = 3$	68	0.9091	0.2878	0.4372
$K = 5$	68	0.9524	0.2878	0.4420
$K = 9$ (default)	68	0.9756	0.2878	0.4444
$K = 15$	68	0.9750	0.2806	0.4358
<i>(C) Transformer mask ratio r</i>				
$r = 0.10$	68	0.1615	0.2230	0.1873
$r = 0.15$ (default)	68	0.1968	0.2662	0.2263
$r = 0.20$	68	0.1711	0.2302	0.1963
$r = 0.30$	68	0.1701	0.2374	0.1982

TABLE VI
F1-SCORE UNDER DIFFERENT GROUPING CONFIGURATIONS ON BGL (SEED 42)

Grouping Configuration	PCA	DeepLog (top-k)	TCN (top-k)
$W = 50$ (non-overlap)	0.1500	0.8556	0.8750
$W = 100$ (non-overlap)	0.1353	0.8571	0.8769
$W = 200$ (non-overlap)	0.4381	0.8810	0.8506
$W = 100$, stride 50 (sliding)	0.2314	0.9004	0.9032

TABLE VII
QUALITATIVE EXPLAINABILITY CASE STUDIES FOR LOG ANOMALIES

Log Set	Seq ID	Pos	Mismatch (Actual)	Event	Top-3 Predicted	NLL	Interpretation	
HDFS	blk_-68076...	4	E19: Receiving empty packet...	E5, E4	E3, E4	12.39	block failure	write
BGL	W20200	1	E18: data TLB interrupt...	E3, E4	E6, E4	12.68	kernel fault	page

resulting in low F1 due to mixing different events); $\theta=0.7$ causes template explosion (2,455 templates, causing F1 to collapse to 0.0446); $\theta=0.5$ is the sweet spot, which is used as the default value throughout the paper. (B) DeepLog top-k saturates around $K=5-9$ with high precision and stable recall; $K=1$ is too strict (precision of 0.05), while $K=15$ begins to miss anomalies. (C) The mask ratio for the mini Transformer achieves the highest F1-score at the default value of $r=0.15$ — aligning with the classic BERT mask ratio [4]; increasing r to 0.20–0.30 reduces effectiveness as too many

TABLE V
PER-SAMPLE INFERENCE LATENCY, BATCH SIZE 1, AVERAGED OVER 1,000 HDFS SAMPLES (UNIT: MS)

Branch	Mean	p95	p99	Throughput
Drain3 parse 1 line	0.007	0.010	0.023	147,458 logs/s
PCA reconstruction	0.063	0.153	0.219	15,869 sequences/s
TruncatedSVD	0.176	0.281	0.381	5,689 sequences/s
Isolation Forest	4.944	9.015	11.396	202 sequences/s
DeepLog (CPU)	1.743	2.086	2.407	574 sequences/s
DeepLog (GPU)	0.489	0.798	0.962	2,046 sequences/s
TCN (CPU)	1.648	2.096	2.609	607 sequences/s
TCN (GPU)	0.455	0.780	0.950	2,198 sequences/s
Transformer (CPU)	2.395	2.966	3.267	418 sequences/s
Transformer (GPU)	2.916	4.323	4.889	343 sequences/s

tokens are masked. The vocabulary growth analysis shows that the number of templates increases from 13 (5k lines) to 137 (500k lines) in a logarithmic pattern, reflecting the template generation characteristics of real system logs.

Expected Error Analysis: Four main types of errors are observed: (i) FP on rare sequences — normal but rare event sequences are incorrectly flagged as anomalies; (ii) FP on long sequences — reconstruction error naturally increases with sequence length, requiring normalization; (iii) FN when anomalies use common templates — requiring sequence models (TCN/DeepLog); (iv) template collision — Drain3 merges different logs due to a loose similarity threshold, requiring fine-tuning of θ .

Inference Latency and Deployment Cost: A real-world log monitoring system must measure warning latency alongside the F1-score. To establish a benchmark, we measure the per-sample inference latency for each branch with a batch size of 1 on an RTX 4050 Laptop GPU (Drain3 and baselines measured on CPU; DeepLog/Transformer measured on both CPU and GPU) with 50 warm-up runs, averaged over 1,000 HDFS samples (Table V).

Three observations from Table V: (1) Drain3 processes ~ 228 k logs/sec, which is not a bottleneck; (2) PCA/SVD are the fastest (< 0.2 ms); IF is $100\times$ slower due to traversing 200 trees; (3) TCN and DeepLog benefit from GPU execution (latency drops from ~ 1.7 ms to 0.45ms), whereas the small Transformer does not benefit at batch size 1. Training times: Drain3 takes 3.49 s, PCA 0.74 s, and Transformer 10.75 s —

a trade-off that needs consideration in real-time monitoring.

C. Applications and Limitations

The pipeline could potentially support detection of brute-force password scanning (repeated login failures), anomalous API utilization (new templates from Drain3), or service-overload patterns (HTTP 5xx sequences) in production networks after validation on corresponding production logs. This is a proposed application analysis that requires confirmation on anonymized production data.

Qualitative Explainability Analysis: To enhance operational utility, we present a qualitative explainability layer based on next-event prediction mismatches. Table VII illustrates two concrete anomaly cases extracted from HDFS and BGL test sets using TCN. When a sequence is flagged, the system outputs the failed event position, the actual versus top-3 predicted event templates, and a severity score (NLL). For instance, in BGL sequence W20200, the unexpected occurrence of E18 (data TLB error interrupt) instead of the highly probable normal transitions (E3, E6, E4) produces a high anomaly score (12.68), immediately guiding operators to a virtual memory subsystem fault.

Threats to Validity: (i) experiments are conducted on standard HDFS/BGL datasets and have not been tested at scale in production environments; (ii) domain shift exists between lab datasets and real logs; (iii) benchmark labels may contain noise; (iv) Drain3 is sensitive to the threshold θ ; (v) performance depends heavily on grouping strategies and threshold percentiles; (vi) the mini Transformer is not deeply optimized; (vii) anonymization and warning validation mechanisms are required in practice.

VI. CONCLUSION

This paper has constructed and evaluated a reproducible log anomaly detection pipeline based on Drain3 and event sequence analysis, comparing lightweight baselines and sequence models (TCN, Transformer, LSTM) in parallel on HDFS and BGL across 5 random seeds. A key finding is that no single scoring branch dominates across all settings: PCA remains competitive on short-sequence datasets (HDFS), while TCN shows strong performance on BGL (F1-score of 0.9826 ± 0.0022) with lower inference latency than LSTM and Transformer on CPU. These results suggest that practitioners should select scoring branches based on dataset characteristics. For operational deployment, an explainability layer could report, for each anomalous sequence, the event position where prediction fails, the actual versus top- k predicted templates, and the per-event anomaly score contribution, thereby reducing manual triage effort. Future directions include: (i) validating the pipeline on production system logs; (ii) incorporating semantic embeddings for log templates; (iii) developing the proposed explainability mechanism into a fully automated alert diagnosis module.

REFERENCES

- [1] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proc. ACM CCS*, 2017.
- [2] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log Anomaly Detection via BERT," in *Proc. IJCNN*, 2021.
- [3] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An Online Log Parsing Approach with Fixed Depth Tree," in *Proc. IEEE ICWS*, 2017.
- [4] A. Vaswani et al., "Attention Is All You Need," in *Proc. NeurIPS*, 2017.
- [5] S. He, J. Zhu, P. He, and M. R. Lyu, "Experience Report: System Log Analysis for Anomaly Detection," in *Proc. IEEE ISSRE*, 2016.
- [6] J. Zhu et al., "Tools and Benchmarks for Automated Log Parsing," in *Proc. ICSE*, 2019.
- [7] V. H. Le and H. Zhang, "Log Parsing with Prompt-based Few-shot Learning," in *Proc. ICSE*, 2023.
- [8] T. Zhang, X. Huang, W. Zhao, S. Bian, and P. Du, "LogPrompt: A Log-based Anomaly Detection Framework Using Prompts," in *Proc. IJCNN*, 2023.
- [9] X. Han, S. Yuan, and M. Trabelsi, "LogGPT: Log Anomaly Detection via GPT," in *Proc. IEEE BigData*, 2023.
- [10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly Detection: A Survey," *ACM Comput. Surv.*, vol. 41, no. 3, 2009.
- [11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proc. IEEE ICDM*, 2008.
- [12] J. Zhu, S. He, J. Liu, M. R. Lyu, and P. He, "Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics," in *Proc. IEEE ISSRE*, pp. 355–366, 2023.
- [13] W. Meng et al., "LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs," in *Proc. IJCAI*, pp. 4739–4745, 2019.
- [14] X. Zhang et al., "Robust Log-Based Anomaly Detection on Unstable Log Data," in *Proc. ESEC/FSE*, 2019.
- [15] L. Yang et al., "Semi-Supervised Log-Based Anomaly Detection via Probabilistic Label Estimation," in *Proc. ICSE*, 2021.
- [16] M. Landauer, S. Onder, F. Skopik, and M. Wurzenberger, "Deep Learning for Anomaly Detection in Log Data: A Survey," *Mach. Learn. Appl.*, vol. 12, 2023.
- [17] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. ICLR*, 2019.
- [18] A. Oliner and J. Stearley, "What Supercomputers Say: A Study of Five System Logs," in *Proc. IEEE/IFIP DSN*, 2007.
- [19] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv:1803.01271*, 2018.